

Unit
3

Recurrent Neural Network

Page No. _____
Date _____

Q] Explain difference between fNN & RNN

Points	Feed forward NN	Recurrent NN
1) Connection type.	No recurrent / feedback connection.	has recurrent connection from hidden loop back to hidden.
2) Information flow	One direction only	Bidirectional.
3) Parameter Usage	New weight per layer	Same weight shared across each layer
4) memory	No past memory	Stores info in hidden state.
5) Time	Does not consider order of time	Considers order.
6) Learning	Limited to current step only	Learns patterns
7) Gradient issue	Usually stable	Suffers from vanishing / exploding gradients
8) Application	Classification, regression	NLP, speech.
9) O/p behaviour	depends only on Input	Depends on i/p + previous hidden state.
10) Complexity	Easier	more difficult due to vanishing / exploding gradient.

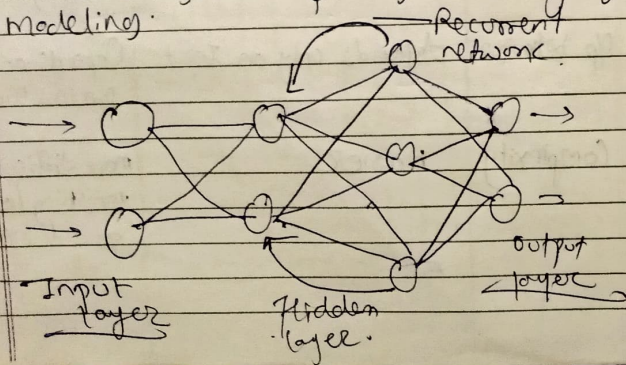
Q] What is recurrent neural network?
 Explain the types of RNN in detail
 → A class of neural network for processing sequential data is called as recurrent neural network (RNN)

- RNN are design to recognize the sequential characteristics in data. & use pattern to predict the next likely scenario

- Unlike other NN the RNN has memory to keep history of past input, this allow it ~~to~~ to make decision by considering current input along with learning from previous input.

- Same parameters & weights are used across time steps, making the model scalable for long sequences.

- RNN is preferred for tasks where order matters, such as ^{speech} recognition, language modeling.



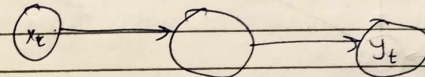
Types of Recurrent neural network

1) One-to-one RNN.

- This neural network is used for fixed sized input to fixed size output for example image classification.

- This type of RNN is also called as vanilla RNN, usually characterized by single token value.

- All traditional neural network fall into this category.



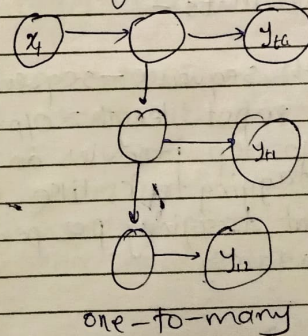
or one-to-one

eg → used in image classification.

2) One-to-many

- A single input is ~~used~~ used to create multiple output.

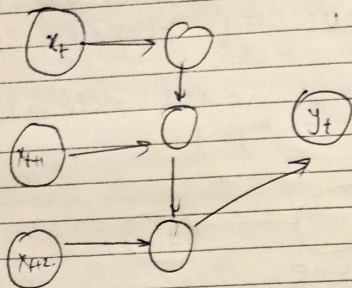
eg → A popular application of this is music generation



one-to-many

3) many-to-one:

- consists of several inputs that used to create a single output.
- In this RNN type all information gets aggregate into last hidden state.



many-to-one.

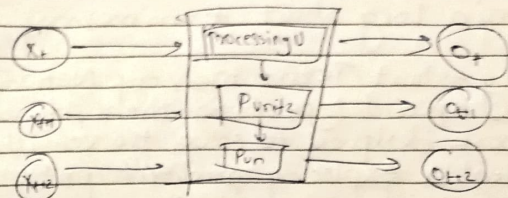
eg → sentiment analysis, document classification.

4) Many-to-many RNN

- several inputs are used for generating several outputs.
- It has two variants -

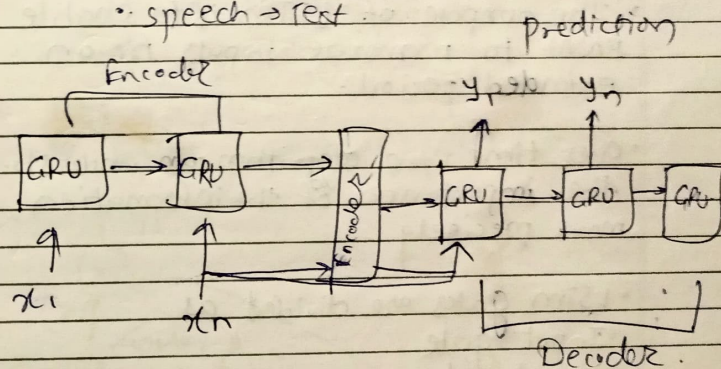
a) Equal length sequence → sequence.

- In this input length = output length.
- Every T/p step produces an output.
- used in tagging tasks like:
 - sentiment tagging per person
 - POS tagging



b) Encoder-Decoder.

- In this $I/p \text{ len} \neq O/p \text{ len}$.
- widely used in:
 - machine translation
 - speech → text



Advantages of RNN

- 1) RNN can process inputs of any length.
- 2) RNN keeps the history of previous inputs.
- 3) Even if T/p size is larger, the model size does not increase.
- 4) Weights stays same at each step.
- 5) RNN can handle variable length sequences.

Q] Explain long short term memory

→ A long-short Term memory Network is a special type of Recurrent Neural Network designed to solve the vanishing gradient & exploding gradient problems that occur in normal RNNs.

LSTM is responsible for memory extension. LSTM forms the building units for the layers of an RNN.

The purpose of LSTM is to enable RNN to memorize inputs for an extended period.

Over time, the algorithm can understand the importance of the information more precisely.

LSTM gates are divided as

- Input gate
- forget gate
- Output gate.

a) Input Gate -

• Input gate decides how much of the current input and hidden information should be added to the state.

• It uses the sigmoid fn.

• It helps in adding fresh info along with previous past.

b) forget Gate

- forget gate decides how much of the previous state stays in the current state.
- It regulates the weight of self loop.
- Its value is betⁿ 0 to 1, generated by sigmoid fⁿ.

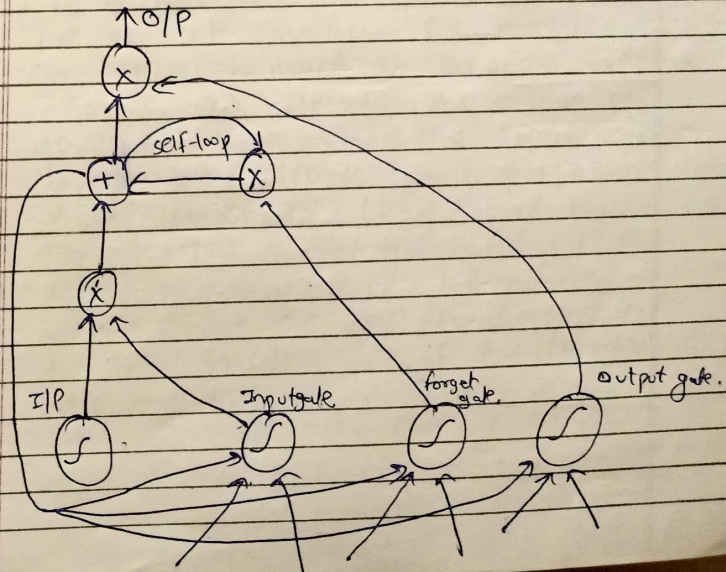
1 → keep old memory

0 → forget previous memory

c) Output gate

- O/p gate decides how much of the internal memory should be sent to the hidden o/p(h⁺).
- It uses sigmoid gate.

$$h_i^{(+)} = \tanh(s_i^{(+)} \times q_i^{(+)})$$



Advantages

- 1) LSTM can remember info for long time period.
- 2) It avoids the vanishing gradient problem.
- 3) It learns long term dependencies.
- 4) more flexible due to gates.

Applications

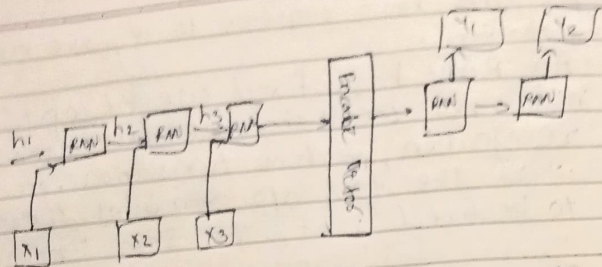
- Speech recognition
- Handwriting generation
- Machine translation
- Image captioning

Q5 Explain how sequence to sequence model works.

→ An RNN has fixed size of input & output. it is not useful in cases like speech recognition, machine translation, etc., where the ip & op sequences do not need to be fixed & of the same length.

• Sequence-to-sequence (seq2seq) models can help in solving above problems. When given an input, the encoder-decoder model first generates an encoded representation of the model, which is then passed to decoder to generate the desired output.

• The idea behind the design of this model is to enable it to process input where we do not constrain the ~~and~~ length. One RNN is used as encoder and another as decoder. The output vector generated by the ^{encoder} and the input vector given to the decoder will possess a fixed size. However they need not to be equal. The output generated by the encoder can either be given as a whole chunk or can be connected to the hidden units of the decoder unit at every time step.



There are three main blocks in the encoder-decoder model:
Encoder, Hidden vector & decoder.

* Encoder *

- Takes the input sequence one time-step at a time.
- Updates its hidden state after each input.
- Encoder vector is last hidden layer.

* Decoder *

- The input for the decoder is the final hidden vector obtained at the end of encoder model.
- Each layer will have three inputs, hidden vector from previous layer, output of previous layer, original hidden vector h .

* Limitation *

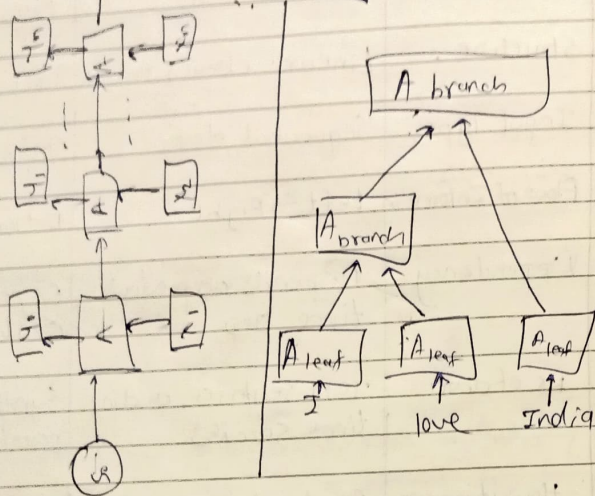
If the encoder context vector is too small, it cannot summarize long

Sequences properly.

Q) Differentiate Recurrent NN & Recursive NN.

Feature	Recurrent NN	Recursive NN
1) Structure.	Linear chain structure	Tree-like structure
2) Input Type.	Sequential data	Structured data.
3) Flow of Information	Left $\rightarrow$ Right	Bottom-up
4) Dependency	Depends on previous time step	Depends on child node
5) Ex of data.	Text sentence, audio, time series	Synthetic tree, parent-child relation
6) How it process	Reads data one step at a time.	Combines sub-parts to form parent representation
7) Application focus	predict next step or classify sequence	Understand structures composition
8) Output	at final node	at root or intermediate node.
9) problem	Gradient & exploding gradients	pruning can be slow.

- 10 When to use
- 11 Example
- When we have sequential data
- Machine translation
- hierarchical data
- Syntactic parsing, Sentiment analysis



3 Describe Recursive Neural Network & type of Recursive Neural Network.

Recursive NN are generalization of recurrent network where the computation graph is arranged in a tree structure instead of a chain.

In a recursive network, input data is organized into a hierarchical tree. The model combines children representations to form parent representations.

This approach is bottom-up unlike RNN that goes from left to right.

Recursive NN are used in computer vision, Natural language processing.

How it works?

Step 1 → Start with Input Node (leaves)

- Inputs are placed at the leaves of a tree.
- These can be
 - words in a sentence
 - parts of an image
 - sub exp in mathematical formula.

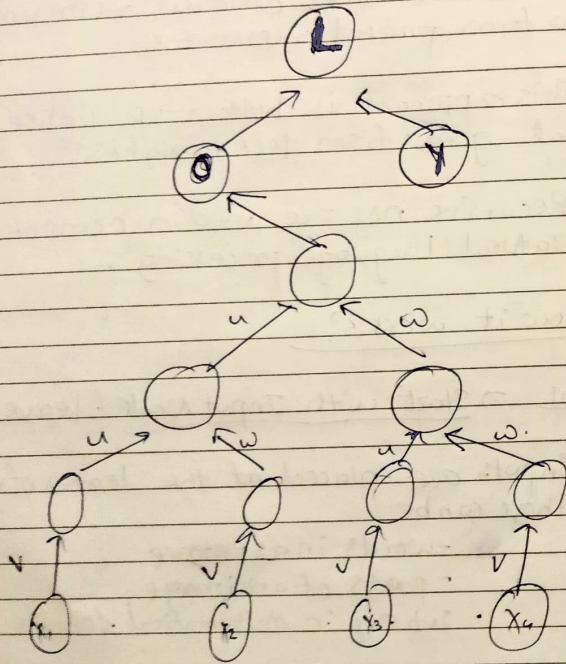
Step 2 → Combine Child nodes into a parent

- A neural F^h is applied to the leaf nodes.
- The result becomes the representation of the parent.

This F^n uses shared weight matrices (U, V, W) .

Step 3. Repeat recursively upwards

- Each new parent node becomes input to the next node.
- finally, the root node of the tree is formed.



eg → word → become ip vector

Two adjacent word → combines to form phrase vector

Two phrase → combine to form sentence vector

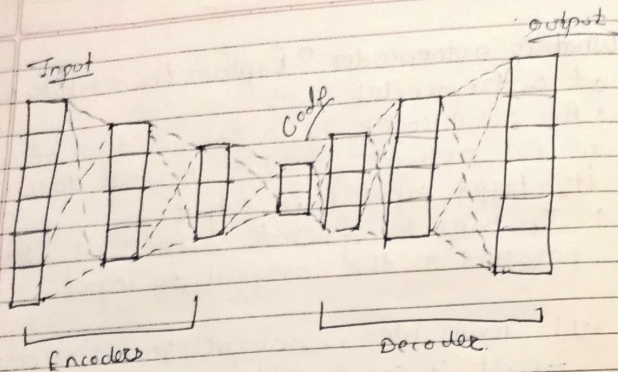
Sentence vector → use for prediction.

Advantages

- 1) RecNN works on tree-like structures also.
- 2) They work with variable-sized input
- 3) They combine small parts into meaningful bigger structures.
- 4) Used in real world use cases like NLP.
- 5) Helps in reasoning & sentimental analysis.

Q] What is autoencoder? Explain the architecture of autoencoder.

-
- An autoencoder is a special type of a neural network that is trained to copy its ~~type~~ input to its output.
 - It tries to generate output as close as possible to the original ~~an~~ input.
 - Its main idea is compression + reconstruction.
 - first, it compress the data into a smaller hidden representation.
 - Then it reconstruct the input back from that ~~the~~ compress form.
 - Autoencoder is not classifier, it does not predict labels. It learns meaningful low dimensional features automatically from data. These features are called code.
 - It discovers patterns without supervision making it useful for feature extraction, denoising, anomaly detection.
 - Autoencoders belong to the FFNN, where they are trained using backpropagation.
 - During training model tries to minimize reconstruction error, using loss fn like ms



A) Encoder

- 1) Encoder takes the original input & converts it into a smaller feature vector
- 2) It is stack of layer where each next layer has fewer neurons
- 3) Encoder extracts useful information & drops unimportant details.
- 4) Encoder weights are learned during training, to preserve important data pattern

B) Code

- This is centre layer with minimum number of neurons.
- It holds a compressed summary of the input.

- The code is designed to keep only essential features required for reconstruction.
- Smaller the bottleneck more is compression but have more loss
- A larger bottleneck $\rightarrow$ less compression but better reconstruction.

C) Decoder

- Decoder performs reverse operation of encoder.
 - It expands the compact code back to the original dimension.
 - Decoder layers usually increase in size step by step.
 - Goal of decoder is to reproduce output as close as possible to the input.
- # There are four hyperparameter that must be set before training autoencoder
- 1) Code size $\rightarrow$ no of node in middle layer, smaller more compressed
 - 2) Number of layers $\rightarrow$ Autoencoder can be as deep as we like without considering the I/P & o/p
 - 3) Num of nodes per layer: encoder have decreasing order decoder have increasing order
 - 4) Loss fn $\rightarrow$ cross entropy is used if the I/P values are in range (0,1) else MSE is used

(8) Explain Undercomplete Autoencoders.
 • An autoencoder in which dimension of the code is less than dimension of input is called undercomplete encoder.
 $\text{code dimension} < \text{Input dimension}$.

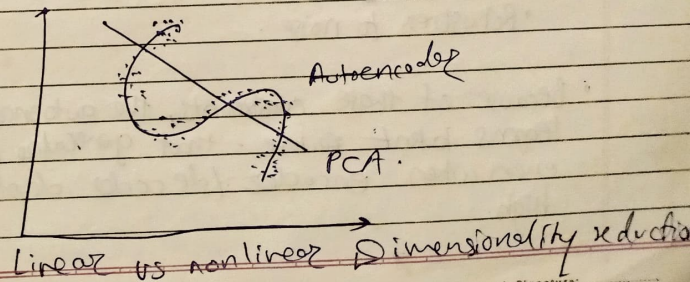
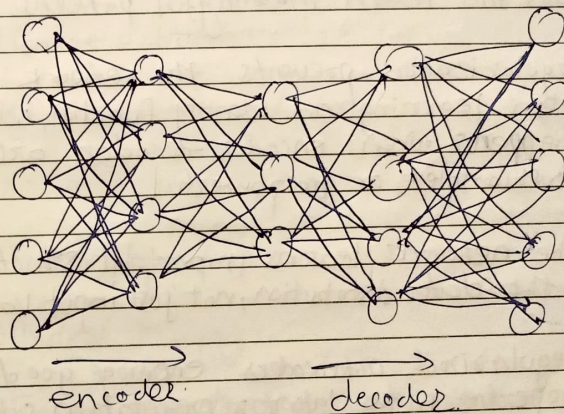
- Because the code is small, the network is forced to compress and keep only useful information.
- This prevents the model from simply copying data & make it learn meaningful patterns.
- The network learn most salient (imp) feature of input.
- The objective is to minimize reconstruction loss, usually with MSE

$$L(x, g(f(x)))$$

$f(x) = \text{encoder}$ & $g(f(x)) = \text{decoder}$

- Undercomplete architecture limits the amount of information flow, which encourages real learning.
- If encoder & decoder are linear & MSE is used, the model behaves like PCA.

- If encoder / decoder are non-linear, it becomes nonlinear generalization of PCA, capturing more complex structure.
- It learns principal features of training data as a side effect of reconstruction.
- It is preferred because it prevents overfitting and learning only memorized values



Q7] What is Regularized Autoencoder?

Where are they used?

→ A regularized auto encoder is an encoder that uses extra constraints in the loss function to force the model to learn useful features instead of just copying the input.

- Unlike undercomplete autoencoder, regularized autoencoders do not depend only on small code size, they can have large capacity but still learn meaningful patterns.
- Regularization prevents the network from learning an identity function, which happens when encoder-decoder are overcomplete or too powerful.
- The network learns important structure of the data distribution, not just input values.
- Regularized autoencoders encourage good properties in the latent representation such as
 - Sparsity
 - Smoothness
 - Robustness to noise.
- Because of those constraints, the autoencoder learns latent feature that generalize better, even when encoder/decoder depth is high.

• A regularized autoencoder can be nonlinear and overcomplete and still learn useful information about the dataset.

• Regularization ~~made~~ helps the model behave properly even when model capacity is large, so it does not collapse to trivial copying.

→ Regularized autoencoders are used in

1) Feature extraction.

2) Dimensionality reduction.

3) Noise robust learning.

4) Anomaly detection.

5) Medical imaging.

6) Image reconstruction & denoising.

Q1] Explain Sparse Autoencoder with its advantages & disadvantages

• Sparse Autoencoder is a type of regularized autoencoder that forces only a small number of hidden neurons to be active at a time.

• Instead of reducing code size, it applies a sparsity constraint to the hidden layer.

• Sparsity constraint means most neurons have near-zero activation features, & only a small subset becomes active for any input.

• The idea is that the ~~neuron~~^{network} must capture important feature using very few neurons, instead of memorizing or copying the I/p.

• A penalty term is added to the loss f to enforce this constraint.

$$L(x, g(f(x))) + \lambda(h)$$

First term = reconstruction error

Second term: $\lambda(h)$ = Sparsity penalty.

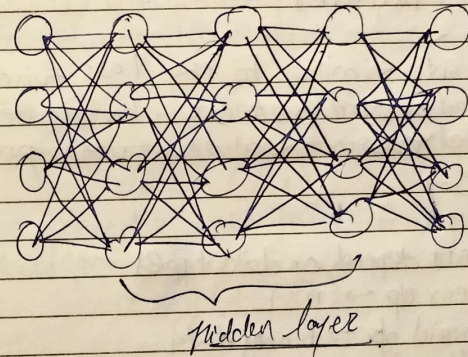
• Sparsity penalty is applied to hidden layer, preventing overfitting & forcing meaningful representation.

1) Sparse autoencoder works well even when hidden layer is larger than the input, because activation is controlled by sparsity, not size.

2) Features learned by sparse autoencoders are often used for other tasks, later, such as classification or clustering.

3) Sparsity Sparse penalty can be treated like a regularization term. added to a feedforward network whose primary goal is reconstruction.

I/p $\xrightarrow{\text{encoder}}$ $\xrightarrow{\text{decoder}}$ o/p layer.



Advantage.

- 1) learn meaningful features
- 2) better feature extraction
- 3) Reduce overfitting.
- 4) powerful in nonlinear scenarios
- 5) Robust feature learning.

Q) Write short note on stochastic autoencoders
Decoders
→ Stochastic autoencoders introduce randomness in encoding & decoding, instead of using fixed outputs.

• They treat the hidden representation as a probability distribution rather than a single point/vector

• The encoder produces a conditional distribution over hidden variables.

$$P_e(h|x)$$

• The decoder ~~pro~~ reconstruct input by sampling from another conditional distribution.

$$P_d(x|h)$$

• This is similar to the loss function in feedforward network, but instead of direct error values, it uses probabilistic outputs.

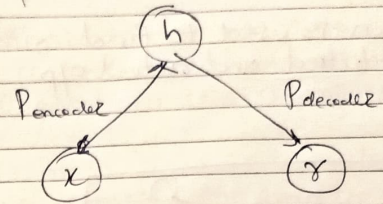
• Op unit depend on data types

• Linear op → MSE

• Sigmoid op → binary data

• Softmax op → categorical distribution

• Given hidden variable h , output dimensions are assumed conditionally independent, making evaluation simpler & computationally cheaper.



Q) Explain denoising autoencoders, In what way, it differs from regular autoencoders? why is it used?

→ • A denoising autoencoder takes a noisy or corrupted input & tries to reconstruct the original clean input.

• Noise is intentionally added to the input data during training.

• The encoder receives the corrupted version and learns features from it.

$$\tilde{x} = C(\tilde{x}|x)$$

• The decoder uses those features to rebuild the clean original input.

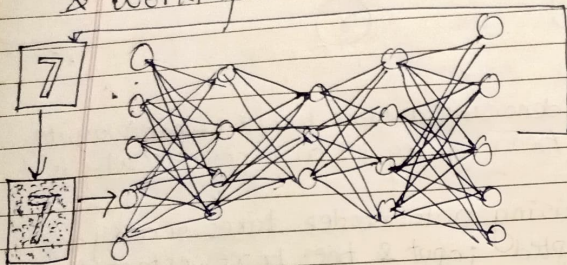
$$x = g(f(\tilde{x}))$$

• This prevents the model from simply copying input & force it to learn meaningful patterns.

• The network learns robust representation, not noise or useless variation.

- Loss function is used to find similarity between predicted and actual o/p.

* Working *



- 1) Takes clean sample x
- 2) Add noise in it make " $\tilde{x}$ " version
- 3) Encode $\tilde{x} \rightarrow h = f(\tilde{x})$
- 4) decode $h \rightarrow g(f(\tilde{x}))$
- 5) Compare $\tilde{x}$ with x using loss f^n .
- 6) Train model to minimize difference.

- Regular Autoencoders have $I/p = 0/p$ (clean = clean)
- While in denoising $I/p = \text{noise}$, $0/p = \text{clean}$
- In Regular Auto- it mimics / copy input while denoising learns to remove noise & recover data
- Regular Autoencoders may not work well with real world data with noise. While denoising vice-versa.

Teacher's Signature:

- 9) Write short note on Contractive Autoencoders
 - A contractive Autoencoder is an autoencoder designed to learn robust & stable feature.
 - Its main goal is to make hidden representation less sensitive to small changes in the input.
 - The penalty term is applied to loss f^n to make the model more robust.
 - In order to make derivatives of f to be as small as possible, CAE introduces explicit regularizer on code $h = f(x)$.
 - Because of this contractive AE learns smooth feature space, not a noisy or unstable one.
 - The penalty used is the Frobenius Norm of the Jacobian matrix, which measures how much the hidden layer changes with respect to the input.
 - In denoising autoencoders, the model only resists big noise added to the input. but in CAE, the model resists very small change. CAE learns better features than using denoising.
 - CAE is generally better for feature extraction.

Teacher's Signature:

because it learns stable patterns.

- Regular AE may memorize input but fails to learn meaningful pattern.

- CAE forces the model to learn new features of dataset, not just copy the Z/p

- It provides simple & better feature extraction than simple autoencoders.

Applications

- 1) Image and pattern recognition.
- 2) Handwriting & digit recognition.
- 3) Medical data analysis.
- 4) Clustering & unsupervised learning.
- 5) Feature extraction for deep ^{neural network} ~~dataset~~

Q.1 State & explain applications of Autoencoders

- 1) Dimensionality reduction.
- 2) Image denoising
- 3) Image Generation
- 4) Data compression
- 5) Feature extraction
- 6) Image Colorization.
- 7) Anomaly detection.
- 8) Data denoising
- 9) Image inpainting
- 10) IR
- 11) Recommendation system

1) Dimensionality reduction

- AE compress high-dimensional Z/p into small hidden layer called the code.
- The hidden layer stores only the imp information, removing unnecessary details.
- Takes high dim. input
- pass it through the encoder
- latent code stores only imp features
- Decoder tries to reconstruct original from this small representation.

2) Image denoising.

- Take original clean image.
- Train noisy image
- Encoder learns pattern of real image
- Decoder reconstructs image
- Loss compares reconstructed ^{with} original.

5) Diff betⁿ ~~in~~ undercomplete or vs sparse or

Feature	VAE	Sparse AE
1) Idea	Reduce hidden layer size to force compression	Hidden layer may be large, but only few neurons active
2) How it learn	Learns imp features bcoz network has limited capacity	Learns important features bcoz most neurons are forced to be inactive
3) Constraint	Capacity constraint	Sparse constraint
4) Hidden layer size	Smaller than I/p size, layer	Equal or larger than I/p
5) Type of regularization	No regularizer	Uses penalty term
5) Activation	All hidden layer	Only a small set of neurons
6) Goal	Compress data & extract core feature	Learn selective feature & ignore noise
7) When useful	Dimensionality reduction	feature learning, anomaly detection
8) Risk	If too small - loses info	Harder to train

Teacher's Signature:

5) Denoising Autoencoder VS Contrastive Autoencoder

Aspect	Denoising AE	cAE
1) Main purpose	make model robust to noise in I/p data	make model robust to small changes in I/p values
2) How it works	Add noise for training	penalizes sensitivity of hidden layer to input variations
3) Input	Noisy / corrupted	Normal I/p
4) Goal	Remove noise & recover clean data	Add penalty to reduce sensitivity
5) Robustness	Through noise injection	Through Jacobian penalty on hidden layer
6) Focus	External corruption	Internal variation in input space
7) Behaviour	Learns to clean input	Learns to code input tightly
8) Use case	Image / audio denoising	strong feature for classification
9) O/p	clean version of I/p	stable latent representation
10) Ex	Recover clean image from blurred version	Two similar I/p's almost same code

Teacher's Signature:

Q] Explain Representation learning

- Representation learning is the process of converting raw data into useful features which a machine learning model can easily understand & use.
- In representation learning, the model automatically learns how to represent data, instead of a human manually designing features.
- A good ~~ex~~ representation makes learning easier. just like representing numbers in Arabic ~~numerals~~ makes arithmetic operations easy compared to Roman numerals.
 $0, 1, 2, 3, 4, \dots, 9$ are easier than $I, II, III, IV, V, \dots, X$.
- Different representation change how difficult a task is. for example, representing a Sorted list in a binary tree allows fast search. $O(\log n)$, while in linked list it is slow ($O(n)$).
- Quality of representation matters in ML. Better features $\rightarrow$ easier classification or prediction.
- FFNN and deep network learn internal representation especially in hidden layers.
eg $\rightarrow$ In image classification, earlier layers learn edges, middle layer learn shapes, & last layer learn object patterns.

hidden
• Last layer is usually useful as it contains most useful features, because it prepares data for the final decision layer (eg. softmax classifier)

• Supervised learning model discovers representation indirectly, while trying to minimize classification loss

• Unsupervised learning methods, directly learn from unlabelled data

• Semi supervised learning learns from both labeled & unlabelled data. This helps to overcome overfitting when labeled data is limited.

• There is trade-off. the model must keep useful information from input but also remove unnecessary or redundant information.

• Humans & animals also learn from limited labeled ~~data~~ examples while learning from past experience this inspires representation learning in AI

Some challenges in representation learning

1) Choosing right representation.

• Not all features are useful

• A bad representation can make learning slow.

2) Tradeoff betⁿ info & abstraction.

• we want to keep imp info.

• But at a same time also want to remove unnecessary information

• Finding right balance is challenging.

3) Overfitting

• Model may memorized training data instead of ~~learning~~ learning general features

• Especially when labeled data is small.

8) Explain Greedy Layer-wise unsupervised pretraining.

→ Deep neural network became successful again because of unsupervised learning, which helped remove the ~~the~~ need for special architectures like CNNs or RNNs during early training.

- Greedy layer-wise pretraining means training the network one layer at a time, starting from the first layer & moving upwards.

- It is called "unsupervised pretraining" because each layer is trained using Unsupervised learning, without label data.

- A single layer model like autoencoder or sparse coding is used to train each layer, & these model learn useful hidden representation.

- The o/p of each trained layer becomes I/p for the next layer, helping every layer learn a better representation step by step.

- This method later makes supervised learning easier, because each layer already understands useful patterns in the data.

• This method avoids the difficulty of training all deep layers together, which was a big problem before the 2000s.

• Hence greedy layer-wise pretraining provides good initialization for training the full deep network, making final learning faster & more stable.

• It is called greedy because each layer learns its own best representation without waiting for other layers.

• The final goal is to fine-tune all layers together after pretraining, using supervised learning to improve accuracy.

Q3 → Explain transfer learning & domain adaptation.
• Transfer learning means using knowledge learned in one task or environment to solve a new but related task. The model does not start with zero it uses what it already knows.

• The idea behind this is that early layers of models learn general features such as edges, shapes, textures, object outlines.

• In transfer learning, we take a pre-trained model & reuse it for a new task with less data. Only last few layers are changed or retrained.

• This helps in reducing or saving the time because model already knows basic patterns. You only guide it to learn the new task.

• It also reduces the need for a large labeled dataset. Most representation is already learned only fine tuning is required.

• Transfer learning improves accuracy because previous learning gives a better feature starting point.

• Instead of random weights we start with meaningful weights.

DATE: / /
PAGE NO.:
Ex → A CNN model is trained on a dataset of cats & dogs (large dataset).
• The model learns edges, ears, fur patterns, shapes, body parts.

Now if you want to classify tigers & wolves
• You take same model & keep early layers (edge & shape detectors).
• You only train final layer to separate tiger vs wolf.

* Domain Adaption

- Domain adaptation is a form of transfer learning where the model keeps the same task, but the input data distribution changes.
- The model is trained in one domain (source domain) & must work in another domain (targeted domain) which looks different.
- Domain shift causes accuracy drop, because the model expects data similar to the training environment but sees different pattern in real use.
- Domain Adaption adjust the model so that its internal features work for the new domain without training from ~~scratch~~ scratch.

• It is useful when collecting data from target domain is expensive or rare.

• It makes learned feature domain invariant so that both domains produce similar internal representation.

• It can be supervised, semisupervised or unsupervised depending on how much target data is labeled.

* There are two types in it.

1) Homogeneous Domain Distribution

- Source & Target data look the same, but comes from different distribution.
- Their Input format is same in both domains. eg → Both sides are imgs. of $28 \times 28 \text{pxl}$.
- Their feature space (Dimensions) is same. eg Both ~~feature~~ dataset have 784 ip features.
- Only the style or environment changes.
 - different lighting
 - different camera
 - different background.
- model structure does not change because input type is same.

• Training is faster because archi. remains same.

eg → Train on handwritten digit images (MNIST)
Test on coloured images (MNIST-M)
Here, only bg. clr. is changed.

2] Heterogeneous Domain ^{Adaptation} Distribution (HEDA)

• Source & Target data are different types or size not just different style.

• Feature space is different.

eg - source has 300 features, target has 1024 features

• I/p type may be different.

• source = image

• target = text.

source = RGB img.

target = Infrared image.

• Model cannot directly transfer weights, because i/p layers mismatch.

• More difficult than homogeneous, because we must transform data or build encoders.

eg → Image is given as I/p
de Text description is asked.

Q] Explain variant of CNN. Explain architecture of DenseNet.

→ • There are number of variants of CNN including ResNet, LeNet, AlexNet, DenseNet etc. we will discuss the DenseNet in detail.

• Compared to ResNet dense connection achieves great accuracy with fewer parameters.

* Dense Block *

• Standard ~~eg~~ convNet uses several convolutions to extract feature/characteristics from the zip picture.

• In Standard CNN passes feature layer-to-layer sequentially (only previous layer is used)

• In ResNet, skip connections add previous o/p back to future layers

• DenseNet uses dense connections.

- every layer takes input from all previous layers.

- Every layer's output is passed to all subsequent layers.

• The connection is done by concatenation, not addition. Each layer receives collective knowledge of all earlier layers.

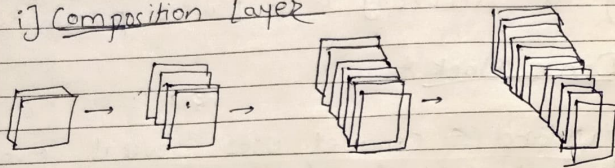
• If each layer adds K new channels, this

is called growth rate (k)

- Because features are reused, network becomes compact & efficient.

DenseNet Architecture

i) Composition Layer

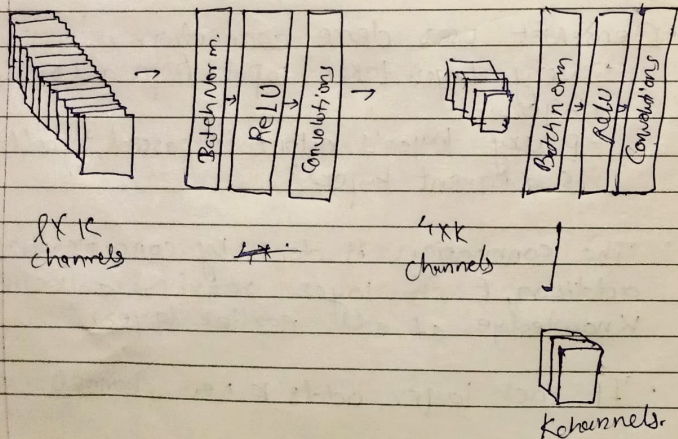


each block performs

- Batch Normalization
- ReLU activation
- 3×3 convolution.

o/p feature maps = k channels (growth rate)

ii) DenseNet-B (Bottleneck Layer)



- To reduce model size, DenseNet uses Bottleneck units.

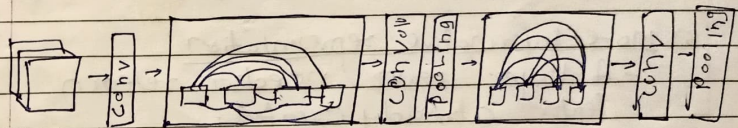
BN $\rightarrow$ ReLU $\rightarrow 1 \times 1$ conv

BN $\rightarrow$ ReLU $\rightarrow 3 \times 3$ conv.

- This decreases computation & model complexity.

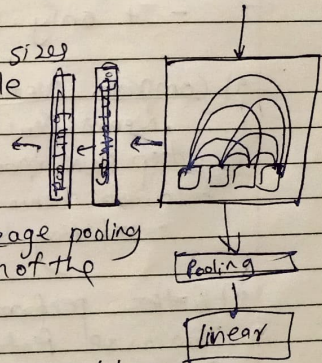
iii) Multiple Dense Blocks with Transition Layers

- Dense blocks are separated using Transition layers.
- The transition layers betⁿ two adjacent dense blocks are 1×1 conv and 2×2 avg pooling.



- within dense block, feature map sizes are uniform, making it simple to concatenate them.

- A softmax classifier is applied once a global average pooling is completed at the conclusion of the final dense block.



- Densenets with/without B/c, various l layers & k growth rates are also trained at this G/p point.

Advantages of Densenet

Why densenet perform well for image classification

1) Efficient Feature use.

- Each layer receives feature maps from all previous layers
- This allows the model to reuse low-level & high-level patterns together.

2) Strong gradient flow.

- This avoids vanishing gradient problem.
- Helps ~~the~~ deeper models train properly
- DenseNet's connection allow error gradients to travel easily to earlier layers

3) More informative representation

- A layer never needs to relearn feature already created.
- It only learns new features.

4) Compact Network

- Needs few parameters compared to other deep CNN models
- Lower memory usage - faster training - less overfitting

5) Better performance with small dataset

- Because features are reused, DenseNet can learn well even when,
- Data is limited.
- Dataset has noise.